

گزارش تمرین دوم درس رایانش سیستم‌های توزیع‌شده

نوید هاشمی 810101549 - سروش اصفهانیان 810101376

در ادامه، گزارش مربوط به پیاده‌سازی سیستم توزیع‌شده مربوط به بخش دوم و سوم تمرین آورده شده است. توجه شود که گزارش مربوط به بخش اول در فایل "rpc_summary" به صورت جداگانه آورده شده است و این گزارش به توضیح پیاده‌سازی مربوط به قسمت دوم و سوم تمرین می‌پردازد.

بخش دوم: پیاده‌سازی سیستم توزیع شده چند ماشین مجازی

پس از بررسی مفاهیم RPC و مقایسه فناوری‌های مختلف در بخش اول، در این قسمت یک سامانه توزیع‌شده ساده با استفاده از زبان Go و فناوری JSON-RPC پیاده‌سازی شد. هدف اصلی این بخش، جداسازی مسئولیت‌ها میان چند ماشین مجازی و برقراری ارتباط شبکه‌ای واقعی میان سرویس‌ها بود؛ به گونه‌ای که هر بخش از سیستم روی ماشین مستقلی اجرا شود و ارتباط اجزا صرفاً از طریق شبکه و نه از طریق فراخوانی محلی با استفاده از localhost انجام گیرد.

معماری نهایی سیستم شامل سه ماشین مجازی اصلی است. ماشین اول (VM1) نقش سرویس وب و رابط کاربری را بر عهده دارد. ماشین دوم (VM2) مسئول احراز هویت کاربران است و به صورت یک سرویس RPC مستقل اجرا می‌شود. ماشین سوم (VM3) نیز نقش مخزن فایل و تصاویر را دارد و فایل‌های مورد نیاز را از طریق شبکه در اختیار سرویس وب قرار می‌دهد.

در این معماری، کاربر از طریق مرورگر به سرویس وب موجود روی VM1 متصل می‌شود. پس از وارد کردن نام کاربری و گذرواژه، سرویس وب درخواست احراز هویت را از طریق JSON-RPC به VM2 ارسال می‌کند. سرویس احراز هویت اطلاعات کاربر را بررسی کرده و نتیجه را به VM1 بازمی‌گرداند. در صورت موفقیت‌آمیز بودن ورود، سرویس وب تصویر ذخیره‌شده روی VM3 را از طریق پروتکل HTTP File Server دریافت کرده و در صفحه نمایش می‌دهد.

دلیل انتخاب JSON-RPC

در میان فناوری‌های مختلف RPC، در این پروژه از JSON-RPC استفاده شد. مهم‌ترین دلیل این انتخاب، سادگی پیاده‌سازی، خوانایی پیام‌ها و عدم نیاز به تعریف فایل‌های واسط اضافی بود. از آنجا که هدف اصلی تمرین آشنایی عملی با معماری RPC و ارتباط میان سرویس‌ها بود، استفاده از JSON-RPC باعث شد تمرکز اصلی ما روی منطق سیستم و ارتباط میان ماشین‌ها قرار داشته باشد و پیچیدگی‌های دیگر مربوط به پروتکل کاهش یابد. علاوه بر این، پیام‌های JSON به راحتی قابل مشاهده هستند و در زمان توسعه امکان بررسی دقیق داده‌های ارسالی و دریافتی را فراهم می‌کنند که خود پوییت مثبت دیگری برای این پروتکل است.

پیاده‌سازی سرویس احراز هویت

سرویس احراز هویت روی VM2 پیاده‌سازی شد. این سرویس به صورت یک JSON-RPC Server اجرا می‌شود و مسئول اعتبارسنجی اطلاعات ورود کاربران است. برای این منظور، فهرست کاربران در یک فایل JSON به نام

users.json ذخیره شده و هنگام راه‌اندازی سرویس در حافظه بارگذاری می‌شود.

همچنین در نسخه نهایی پروژه، گذرواژه‌ها به صورت Hash شده با الگوریتم bcrypt ذخیره می‌شوند. در زمان ورود، گذرواژه وارد شده توسط کاربر با مقدار Hash ذخیره‌شده مقایسه می‌شود و در صورت تطابق، نتیجه موفقیت‌آمیز به سرویس وب بازگردانده می‌شود.

نمونه‌ای از ساختار json مربوط به نام کاربری و پسورد کاربران به صورت زیر است:

```
{
  "username": "alice",
  "password": "$2a$10$Yg3WHehFzImgjsc07VTN20tAy5JWQRY9SKaPD30xzdGez422Qk0bW"
},
```

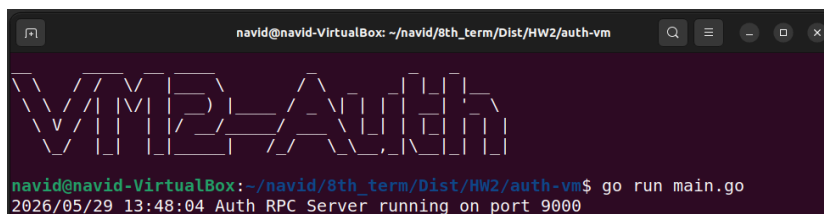
متد اصلی RPC در این سرویس، Login است که اطلاعات نام کاربری و گذرواژه را دریافت کرده و نتیجه اعتبارسنجی را بازمی‌گرداند:

```
func (a *AuthService) Login(req LoginRequest, res *LoginResponse) error {
    for _, user := range a.Users {
        if user.Username == req.Username {
            err := bcrypt.CompareHashAndPassword(
                []byte(user.Password),
                []byte(req.Password),
            )

            if err == nil {
                res.Success = true
                res.Message = "Login successful"
                return nil
            }
        }
    }

    res.Success = false
    res.Message = "Invalid credentials"
    return nil
}
```

برای اجرای این ماشین مجازی و آماده‌سازی آن برای دریافت درخواست از طرف سرور وب نیز تنها کافی است آن را در با استفاده از دستور `go run main.go` اجرا کنیم:



```
navid@navid-VirtualBox: ~/navid/8th_term/Dist/HW2/auth-vm
VM2-Auth
navid@navid-VirtualBox:~/navid/8th_term/Dist/HW2/auth-vm$ go run main.go
2026/05/29 13:48:04 Auth RPC Server running on port 9000
```

پیاده‌سازی سرویس وب

سرویس وب روی VM1 اجرا می‌شود و نقش نقطه ورود کاربران به سیستم را بر عهده دارد. این سرویس با استفاده از کتابخانه استاندارد net/http در Go توسعه داده شده و شامل یک فرم ساده ورود است. پس از دریافت اطلاعات کاربر، سرویس وب یک اتصال JSON-RPC به سرویس احراز هویت برقرار کرده و متد Login را فراخوانی می‌کند. در

صورت موفقیت‌آمیز بودن عملیات احراز هویت، صفحه موفقیت به کاربر نمایش داده می‌شود و فایل مربوطه از سرور فایل دریافت شده و نمایش داده می‌شود. نتیجه ورود موفقیت‌آمیز کاربر و دریافت فایل در قسمت سناریوی تست در ادامه گزارش آورده شده است.

برای نمایش فرم مربوط به صفحه لاگین و همچنین نمایش پیام موفقیت‌آمیز بودن ورود کاربر نیز از دو فایل html به نام‌های login.html و success.html استفاده شده که هر کدام در مود مربوط به خود پارس و نمایش داده می‌شوند.

همچنین برای برقراری ارتباط با سرویس‌های احراز هویت، فایل و subscriber (مربوط به بخش سوم)، نیاز به ip مربوط به هر کدام از این ماشین‌های مجازی می‌باشد که با استفاده از دستور ip a این مقادیر مشخص شده و در فایل main.go مربوط به سرویس وب گنجانده شده اند. آبی هر کدام از ماشین‌های مجازی به صورت زیر است که در فایل go سرور وب گنجانده شده اند:

```
auth server:    192.168.1.111
file server:    192.168.1.108
web server:     192.168.1.110
subscriber (section 3): 192.168.1.100
```

برای اجرای این سرور نیز تنها کافی است آن را با همان دستور قبلی اجرا کنیم:

```
navid@navid-VirtualBox: ~/navid/8th_term/Dist/HW2/web-vm
VM-WebServer
navid@navid-VirtualBox:~/navid/8th_term/Dist/HW2/web-vm$ go run main.go
2026/05/29 13:45:24 Web server running on :8080
```

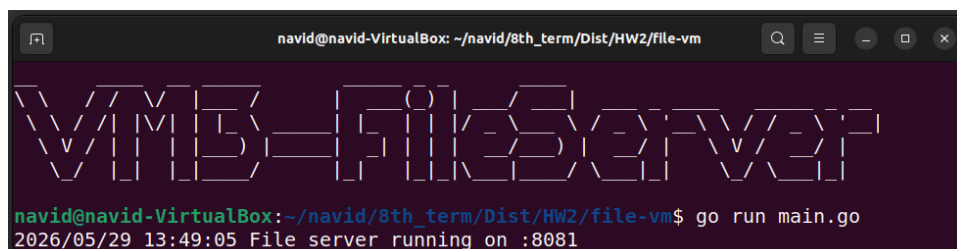
پیاده‌سازی سرویس فایل

ماشین سوم سیستم نقش مخزن فایل و تصاویر را بر عهده دارد. برای این منظور، یک File Server ساده با استفاده از HTTP روی VM3 پیاده‌سازی شد. فایل‌ها و تصاویر مورد نیاز سیستم در این ماشین در فولدر files ذخیره شده‌اند و از طریق یک مسیر مشخص در اختیار سایر اجزا قرار می‌گیرند. برای تست درستی اجرای سیستم از عکس زیر با آدرس files/image.png در ماشین مجازی سرویس فایل استفاده شده است:



نکته مهم این است که تصویر نمایش داده شده در صفحه موفقیت روی VM1 ذخیره نشده است. مرورگر کاربر تصویر را از طریق آدرس شبکه‌ای مربوط به VM3 دریافت می‌کند. به این ترتیب شرط اصلی تمرین مبنی بر دریافت فایل از طریق شبکه و عدم استفاده از مسیرهای محلی رعایت شده است.

حال برای شروع کار این سرویس نیز نیاز است تا آن را اجرا کنیم:



```
navid@navid-VirtualBox: ~/navid/8th_term/Dist/HW2/file-vm
VM3-FileServer
navid@navid-VirtualBox:~/navid/8th_term/Dist/HW2/file-vm$ go run main.go
2026/05/29 13:49:05 File server running on :8081
```

بخش سوم: افزودن سازوکار Publish/Subscribe و اعلان مصرف حافظه

در این بخش یک سازوکار ساده Publish/Subscribe به سامانه اضافه شد. هدف از این قسمت، پایش مصرف حافظه سرویس وب و تولید هشدار در صورت عبور مصرف حافظه از یک آستانه مشخص بود.

برای پیاده‌سازی این بخش، خود سرویس وب نقش Publisher را بر عهده خواهد داشت و یک برنامه مستقل روی ماشین مجازی چهارم به عنوان subscriber اجرا خواهد شد. ارتباط میان این دو مؤلفه نیز از طریق TCP برقرار می‌شود.

طراحی سازوکار Publish/Subscribe

در این پیاده‌سازی، سرویس وب به صورت دوره‌ای میزان مصرف حافظه خود را بررسی می‌کند. در صورتی که مصرف حافظه از مقدار تعیین‌شده بیشتر شود، یک رخداد جدید تولید شده و برای subscriber ارسال می‌شود.

subscriber روی ماشین مجازی مستقل اجرا شده و منتظر دریافت پیام‌های منتشرشده از طرف Publisher باقی می‌ماند. پس از دریافت رخداد، پیام هشدار روی خروجی برنامه نمایش داده می‌شود.

برای انتقال رخدادهای JSON استفاده شده است. قالب کلی پیام منتشرشده به صورت زیر است که در صورت پروژه نیز قید شده بود:

```
{
  "event_type": "HIGH_MEMORY_USAGE",
  "service": "web-server",
  "memory_mb": 345,
  "threshold_mb": 300,
  "timestamp": "2026-..."
}
```

پایش مصرف حافظه

برای پایش مصرف حافظه از امکانات داخلی زبان Go و ساختار runtime.MemStats استفاده شد. در بازه‌های زمانی مشخص، میزان حافظه تخصیص‌یافته توسط برنامه اندازه‌گیری می‌شود و در صورت عبور از آستانه تعریف‌شده، رویداد هشدار تولید می‌شود. مطابق صورت تمرین، مقدار 300 مگابایت به عنوان آستانه مصرف حافظه انتخاب شد. این پیاده‌سازی در زیر قابل مشاهده است:

```
func monitorMemory() {
    for {
        var mem runtime.MemStats

        runtime.ReadMemStats(&mem)

        memoryMB := mem.Alloc / 1024 / 1024

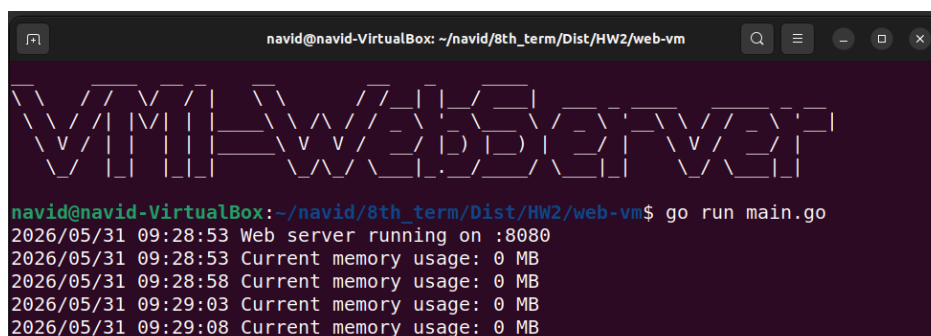
        log.Printf("Current memory usage: %d MB\n", memoryMB)

        if memoryMB > memoryThresholdMB {
            event := MemoryEvent{
                EventType: "HIGH_MEMORY_USAGE",
                Service: "web-server",
                MemoryMB: memoryMB,
                Threshold: memoryThresholdMB,
                Timestamp: time.Now().Format(time.RFC3339),
            }

            publishEvent(event)
        }

        time.Sleep(5 * time.Second)
    }
}
```

در زیر نمونه‌ای از خروجی پایش حافظه در هنگام راه‌اندازی سرویس وب مشاهده می‌شود که با توجه به اینکه هنوز حافظه‌ای اختصاص داده نشده است، مقدار 0 نمایش داده شده است:



```
navid@navid-VirtualBox: ~/navid/8th_term/Dist/HW2/web-vm
VMWARE
navid@navid-VirtualBox:~/navid/8th_term/Dist/HW2/web-vm$ go run main.go
2026/05/31 09:28:53 Web server running on :8080
2026/05/31 09:28:53 Current memory usage: 0 MB
2026/05/31 09:28:58 Current memory usage: 0 MB
2026/05/31 09:29:03 Current memory usage: 0 MB
2026/05/31 09:29:08 Current memory usage: 0 MB
```

افزایش عمدی مصرف حافظه

برای آزمایش عملکرد سیستم هشدار، یک Endpoint با نام consume-memory پیاده‌سازی شد. این Endpoint امکان تخصیص عمدی حافظه را فراهم می‌کند تا بتوان رفتار سیستم را تحت بار مصنوعی اضافه شده بررسی کرد. یک نمونه از این فراخوانی می‌تواند به صورت http://WEB_VM_IP:8080/consume-memory?mb=100 در مرورگر سیستم میزبان باشد که میزان حافظه تخصیص داده شده در سرویس وب را به میزان 100 مگابایت افزایش می‌دهد.

ساز و کار این افزایش حافظه به این صورت است که در هر بار فراخوانی این endpoint، مقدار مشخصی حافظه تخصیص داده شده و در یک ساختار سراسری نگهداری می‌شود تا توسط Garbage Collector آزاد نشود. به این ترتیب می‌توان مصرف حافظه برنامه را به صورت کنترل‌شده افزایش داد:

```
func consumeMemoryHandler(w http.ResponseWriter, r *http.Request) {
    mbStr := r.URL.Query().Get("mb")

    if mbStr == "" {
        mbStr = "100"
    }

    mb, err := strconv.Atoi(mbStr)
    if err != nil {
        http.Error(w, "Invalid mb parameter", 400)
        return
    }

    block := make([]byte, mb*1024*1024)

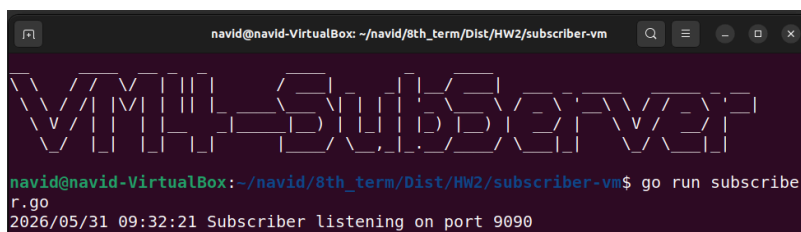
    for i := range block {
        block[i] = 1
    }

    memoryHog = append(memoryHog, block)

    fmt.Fprintf(w,
        "Allocated %d MB memory\nCurrent allocations: %d\n",
        mb,
        len(memoryHog),
    )
}
```

تولید و دریافت هشدار

پس از عبور مصرف حافظه از مقدار آستانه، سرویس وب یک رخداد HIGH_MEMORY_USAGE تولید می‌کند. این رخداد از طریق TCP برای Subscriber ارسال می‌شود. Subscriber پس از دریافت پیام، اطلاعات هشدار را نمایش می‌دهد. این اطلاعات شامل نام سرویس، مقدار مصرف حافظه، آستانه تعریف‌شده و زمان وقوع رخداد است که یک نمونه از آن در سناریو تست قابل مشاهده است. برای راه‌اندازی subscriber نیز آن را با دستور go run اجرا می‌کنیم:



```
navid@navid-VirtualBox: ~/navid/8th_term/Dist/HW2/subscriber-vm
VIM-Subscriber
navid@navid-VirtualBox:~/navid/8th_term/Dist/HW2/subscriber-vm$ go run subscribe
r.go
2026/05/31 09:32:21 Subscriber listening on port 9090
```

آزمایش عملکرد کل سیستم و سناریوی تست

برای ارزیابی عملکرد سامانه، سناریوی کامل ورود کاربر و دریافت و فایل و سپس دریافت اعلان مصرف حافظه توسط subscriber به طور کامل اجرا شده و برای هر مرحله از سناریوی موجود در صورت تمرین، عکس مربوط به آن در ادامه قرار داده شده است. نمایش خطا در هنگام ورود اطلاعات نادرست، نمایش صحیح فایل و دریافت اعلان مصرف حافظه نشان از درستی عملکرد سیستم دارد.

1. نمایش آدرس و IP هر ماشین مجازی

```
navid@navid-VirtualBox: ~/navid/8th_term/Dist/HW2
VM1-Auth
navid@navid-VirtualBox:~/navid/8th_term/Dist/HW2$ ip a | grep "inet"
inet 127.0.0.1/8 scope host lo
inet6 ::1/128 scope host
inet 192.168.1.110/24 brd 192.168.1.255 scope global dynamic noprefixroute e
np0s3
inet6 fe80::de2e:185d:5d02:e3e9/64 scope link noprefixroute
inet 192.168.122.1/24 brd 192.168.122.255 scope global virbr0
inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
navid@navid-VirtualBox:~/navid/8th_term/Dist/HW2$
```

```
navid@navid-VirtualBox: ~/navid/8th_term/Dist/HW2/auth-vm
VM2-Auth
navid@navid-VirtualBox:~/navid/8th_term/Dist/HW2/auth-vm$ ip a | grep "inet"
inet 127.0.0.1/8 scope host lo
inet6 ::1/128 scope host
inet 192.168.1.111/24 brd 192.168.1.255 scope global dynamic noprefixroute e
np0s3
inet6 fe80::5aaa:a843:279f:874f/64 scope link noprefixroute
inet 192.168.122.1/24 brd 192.168.122.255 scope global virbr0
inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
```

```
navid@navid-VirtualBox: ~/navid/8th_term/Dist/HW2/file-vm
VM3-FileServer
navid@navid-VirtualBox:~/navid/8th_term/Dist/HW2/file-vm$ ip a | grep "inet "
inet 127.0.0.1/8 scope host lo
inet 192.168.1.108/24 brd 192.168.1.255 scope global dynamic noprefixroute e
np0s3
inet 192.168.122.1/24 brd 192.168.122.255 scope global virbr0
inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
```

```
navid@navid-VirtualBox: ~/navid/8th_term/Dist/HW2/subscriber-vm
VM4-Subscriber
navid@navid-VirtualBox:~/navid/8th_term/Dist/HW2/subscriber-vm$ ip a | grep "ine
t"
inet 127.0.0.1/8 scope host lo
inet6 ::1/128 scope host
inet 192.168.1.100/24 brd 192.168.1.255 scope global dynamic noprefixroute e
np0s3
inet6 fe80::d6e1:2f18:c68e:851f/64 scope link noprefixroute
inet 192.168.122.1/24 brd 192.168.122.255 scope global virbr0
inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
```

2. اجرای سرویس احراز هویت در VM2

```
navid@navid-VirtualBox: ~/navid/8th_term/Dist/HW2/auth-vm
VM2-Auth
navid@navid-VirtualBox:~/navid/8th_term/Dist/HW2/auth-vm$ go run main.go
2026/05/29 13:48:04 Auth RPC Server running on port 9000
```

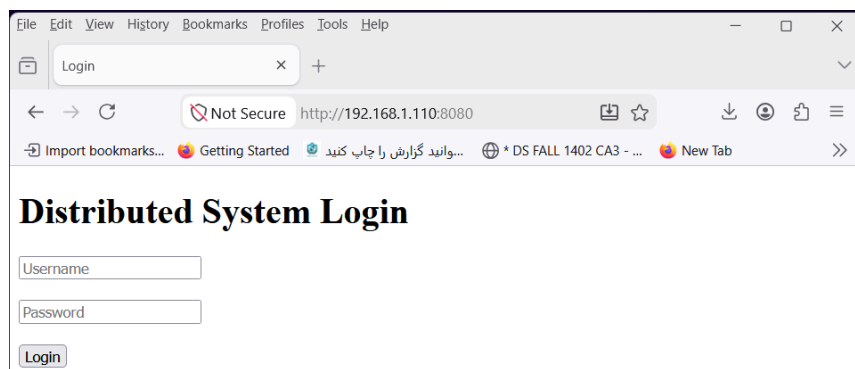
3. اجرای سرویس فایل در VM3

```
navid@navid-VirtualBox: ~/navid/8th_term/Dist/HW2/file-vm
VM3-FileServer
navid@navid-VirtualBox:~/navid/8th_term/Dist/HW2/file-vm$ go run main.go
2026/05/29 13:49:05 File server running on :8081
```

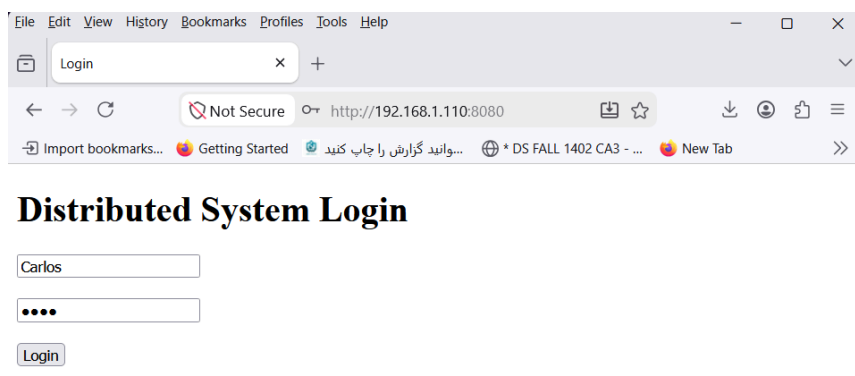
4. اجرای سرویس وب در VM1

```
navid@navid-VirtualBox: ~/navid/8th_term/Dist/HW2/web-vm
VM1-WebServer
navid@navid-VirtualBox:~/navid/8th_term/Dist/HW2/web-vm$ go run main.go
2026/05/31 09:28:53 Web server running on :8080
2026/05/31 09:28:53 Current memory usage: 0 MB
2026/05/31 09:28:58 Current memory usage: 0 MB
2026/05/31 09:29:03 Current memory usage: 0 MB
2026/05/31 09:29:08 Current memory usage: 0 MB
```

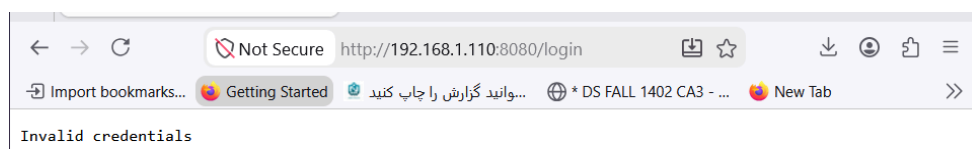
5. باز کردن صفحه وب از سیستم میزبان یا یک کلاینت دیگر



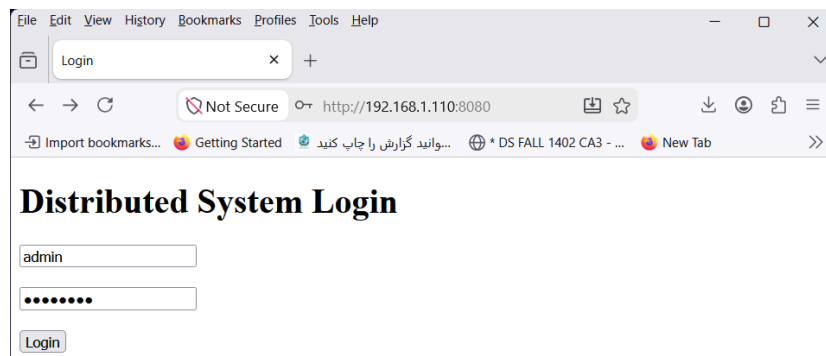
6. تلاش برای ورود با اطلاعات نادرست



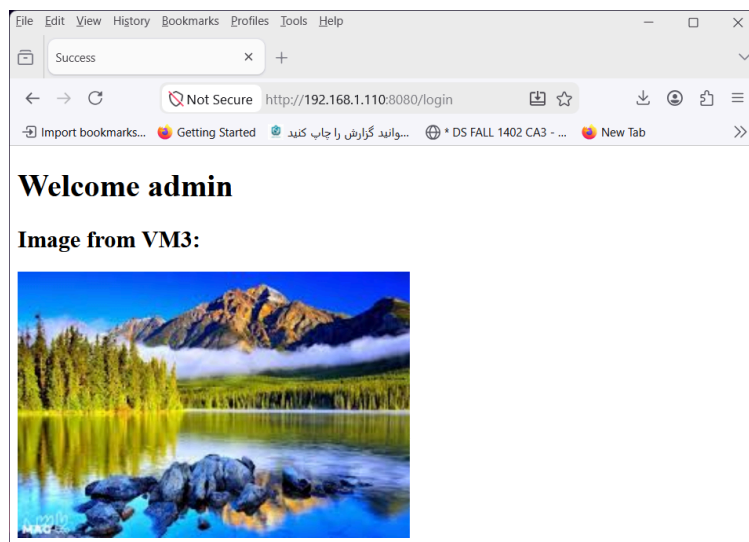
7. مشاهده پیام خطای ورود



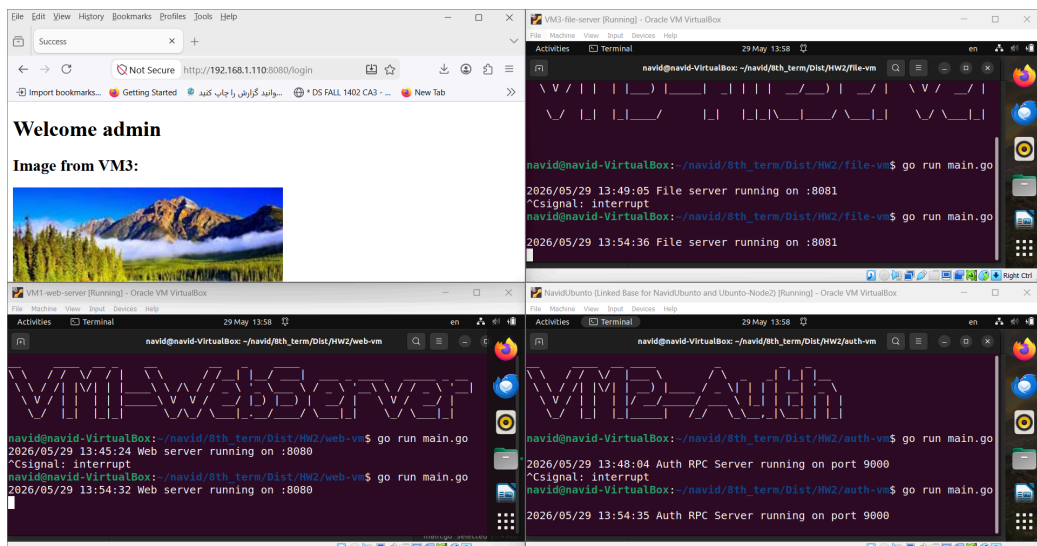
8. تلاش برای ورود با اطلاعات درست



9. مشاهده صفحه موفقیت و 10. دریافت یا نمایش فایل یا تصویر از VM3



نمای کلی اجرای این سه ماشین مجازی تا به اینجا به صورت زیر است:



11. اجرای subscriber

```
navid@navid-VirtualBox: ~/navid/8th_term/Dist/HW2/subscriber-vm
VM-Subscriber
navid@navid-VirtualBox:~/navid/8th_term/Dist/HW2/subscriber-vm$ go run subscribe
r.go
2026/05/31 09:32:21 Subscriber listening on port 9090
```

12. افزایش عمدی مصرف حافظه سرویس وب

```
192.168.1.110:8080/consume-mem: X
Not Secure http://192.168.1.110:8080/consume-memory?mb=100
Allocated 100 MB memory
Current allocations: 1
```

```
navid@navid-VirtualBox: ~/navid/8th_term/Dist/HW2/web-vm
navid@navid-VirtualBox:~/navid/8th_term/Dist/HW2/web-vm$ go run main.go
2026/05/31 09:28:53 Web server running on :8080
2026/05/31 09:28:53 Current memory usage: 0 MB
2026/05/31 09:28:58 Current memory usage: 0 MB
2026/05/31 09:29:03 Current memory usage: 0 MB
2026/05/31 09:29:08 Current memory usage: 0 MB
2026/05/31 09:32:20 Current memory usage: 0 MB
2026/05/31 09:32:25 Current memory usage: 0 MB
2026/05/31 09:32:30 Current memory usage: 0 MB
2026/05/31 09:32:35 Current memory usage: 0 MB
2026/05/31 09:32:40 Current memory usage: 0 MB
2026/05/31 09:32:45 Current memory usage: 0 MB
2026/05/31 09:32:50 Current memory usage: 0 MB
2026/05/31 09:32:55 Current memory usage: 0 MB
2026/05/31 09:33:00 Current memory usage: 0 MB
2026/05/31 09:33:05 Current memory usage: 100 MB
2026/05/31 09:33:10 Current memory usage: 100 MB
2026/05/31 09:33:15 Current memory usage: 100 MB
2026/05/31 09:33:20 Current memory usage: 100 MB
2026/05/31 09:33:25 Current memory usage: 100 MB
```

```
192.168.1.110:8080/consume-mem: X
Not Secure http://192.168.1.110:8080/consume-memory?mb=400
Allocated 400 MB memory
Current allocations: 2
```

```
2026/05/31 09:33:51 Current memory usage: 100 MB
2026/05/31 09:33:56 Current memory usage: 100 MB
2026/05/31 09:34:01 Current memory usage: 500 MB
2026/05/31 09:34:01 Memory event published
2026/05/31 09:34:06 Current memory usage: 500 MB
2026/05/31 09:34:06 Memory event published
2026/05/31 09:34:11 Current memory usage: 500 MB
2026/05/31 09:34:11 Memory event published
2026/05/31 09:34:16 Current memory usage: 500 MB
2026/05/31 09:34:16 Memory event published
2026/05/31 09:34:21 Current memory usage: 500 MB
2026/05/31 09:34:21 Memory event published
```

13. مشاهده هشدار مصرف حافظه بالا

```
navid@navid-VirtualBox: ~/navid/8th_term/Dist/HW2/subscriber-vm
Timestamp: 2026-05-31T09:34:21+01:00
=====
HIGH MEMORY USAGE ALERT RECEIVED
Service: web-server
Memory Usage: 500 MB
Threshold: 300 MB
Timestamp: 2026-05-31T09:34:27+01:00
=====
HIGH MEMORY USAGE ALERT RECEIVED
Service: web-server
Memory Usage: 500 MB
Threshold: 300 MB
Timestamp: 2026-05-31T09:34:32+01:00
=====
HIGH MEMORY USAGE ALERT RECEIVED
Service: web-server
Memory Usage: 500 MB
Threshold: 300 MB
Timestamp: 2026-05-31T09:34:37+01:00
=====
```

همانطور که مشاهده می‌شود، خروجی‌های هر کدام از ماشین‌های مجازی مطابق انتظار بوده که درستی عملکرد سیستم توزیع‌شده پیاده‌سازی‌شده را تایید می‌کنند.